

RTL Design Sherpa

DDR2 / LPDDR2 Family Controller Micro-Architecture Specification 0.1

June 13, 2026

Table of Contents

1 Ddr2 Lpddr2 Mas Index.....	12
2 Document Information.....	12
2.1 DDR2 / LPDDR2 Family Controller Micro-Architecture Specification.....	12
2.2 Document Purpose.....	12
2.3 Document Scope.....	13
2.4 Revision History.....	13
3 Architecture and Datapath.....	14
3.1 Top-Level FUB Topology.....	14
3.2 Read / Write Datapath: AXI → addr-hash → CAM.....	15
3.2.1 Why Two CAMs, Not One.....	15
3.2.2 Why the XOR Happens Before the CAM.....	17
3.2.3 Why the AXI-side metadata stays in axi4_slave_fub.....	17
3.3 Clocking Topology.....	17
3.4 DFI Phase / Gear Topology.....	18
4 Top-Level Port List.....	18
4.1 Parameter-Dependent Widths (re-stated).....	19
4.2 AXI4 Slave Port List.....	19
4.3 DFI v2.1 Master Port List.....	19
4.4 APB CSR Slave Port List.....	19
4.5 Clocks, Resets, IRQs, Debug.....	20
4.6 SystemVerilog Header Snippet.....	20
5 Clocks and Reset.....	21

5.1 Clock Domains.....	21
5.2 Reset Topology.....	21
5.3 CDC.....	22
5.4 Quiet Points.....	22
5.5 Reset Sequence.....	23
6 Top-Level Integration (ddr2_lpddr2_ctrl).....	23
6.1 Purpose.....	23
6.2 Instantiation Inventory.....	24
6.3 Generate Blocks.....	25
6.4 What the Top Does Not Do.....	25
7 AXI4 Slave (axi4_slave_fub).....	26
7.1 Purpose.....	26
7.2 External Interface (AXI side).....	26
7.3 Internal Buffers.....	26
7.4 Data Flow.....	27
7.4.1 Write Path.....	27
7.4.2 Read Path.....	27
7.5 Outstanding Burst Tracking.....	28
7.6 Timing Budget.....	28
7.7 CSR Hooks.....	29
7.8 TODO (week-of-MAS work).....	29
8 Address XOR / Hash (addr_mapper_fub).....	29
8.1 Purpose.....	29
8.2 Mirror to the Python AddressMapping Class.....	30

8.3 Interface.....	30
8.4 Scheme Decode.....	30
8.4.1 Scheme 1: ROW_MAJOR.....	30
8.4.2 Scheme 2: BANK_INTERLEAVE.....	31
8.4.3 Scheme 3: XOR_HASH.....	31
8.5 Runtime Scheme Mux.....	31
8.6 Timing Budget.....	31
8.7 Verification.....	32
8.8 TODO.....	32
9 Read Command CAM (rd_cmd_cam_fub).....	32
9.1 Purpose.....	33
9.2 Parameters.....	33
9.3 Storage Per Slot.....	33
9.4 Interfaces.....	34
9.4.1 Push (from axi4_slave_fub).....	34
9.4.2 Scheduler Query.....	34
9.4.3 Issue Notification.....	34
9.4.4 Beat Return (from rd_data_path_fub).....	35
9.5 Lookup Mechanism.....	35
9.6 Per-ID Beat Counting.....	35
9.7 Timing Budget.....	35
9.8 CSR Hooks.....	36
9.9 TODO.....	36
10 Write Command CAM (wr_cmd_cam_fub).....	36

10.1 Purpose.....	36
10.2 Parameters.....	37
10.3 Storage Per Slot.....	37
10.4 Interfaces.....	38
10.4.1 Push (from axi4_slave_fub).....	38
10.4.2 Scheduler Query.....	38
10.4.3 Issue Notification.....	38
10.4.4 Beat-Pull (to wr_data_path_fub).....	38
10.4.5 Completion (from DFI write window expiration).....	38
10.5 Difference From Read CAM.....	38
10.6 Timing Budget.....	39
10.7 CSR Hooks.....	39
10.8 TODO.....	39
11 Transaction Queue (txn_queue_fub).....	40
11.1 Relationship to the CAMs.....	40
11.2 TODO.....	40
12 FR-FCFS Scheduler (scheduler_fub).....	40
12.1 Purpose.....	41
12.2 Synthesis Parameters.....	41
12.3 Interface.....	42
12.3.1 Inputs.....	42
12.3.2 Outputs.....	43
12.4 Decision Pipeline.....	44

12.4.1 Stage 1 — Per-Entry Readiness (combinational, N parallel comparators).....	45
12.4.2 Stage 2 — Priority Masking (combinational).....	46
12.4.3 Stage 3 — Selection (combinational priority encoders).....	47
12.4.4 Stage 4 — Command Formation (registered).....	47
12.4.5 Refresh Window Behavior.....	48
12.5 Cache Coherency: row_hit_cached.....	48
12.6 Pipeline Staging.....	49
12.7 Strict In-Order Mode (collapse).....	49
12.8 Critical-Path Analysis (rough budget at 500 MHz target, 2 ns cycle).....	50
12.9 CSR Hooks.....	50
12.10 Verification Notes (cocotb test plan).....	51
12.11 Open Questions / Future Work.....	52
13 HAPPY Page Predictor (page_predictor_fub).....	52
13.1 TODO (Week-of-MAS work).....	53
14 Bank Machine (bank_machine_fub).....	53
14.1 TODO (Week-of-MAS work).....	53
15 Cross-Bank Timers (xbank_timers_fub).....	54
15.1 TODO (Week-of-MAS work).....	54
16 Refresh Manager (refresh_mgr_fub).....	54
16.1 TODO (Week-of-MAS work).....	54
17 Init Engine (init_engine_fub).....	55
17.1 TODO (Week-of-MAS work).....	55
18 Power-State FSM (power_state_fub).....	55

18.1 TODO (Week-of-MAS work).....	56
19 Command Encoder (cmd_encoder_fub).....	56
19.1 TODO (Week-of-MAS work).....	56
20 DFI Gear (gear_dfi_fub).....	57
20.1 TODO (Week-of-MAS work).....	57
21 ODT Control (odt_ctrl_fub).....	57
21.1 TODO (Week-of-MAS work).....	57
22 Write Data Path (wr_data_path_fub).....	58
22.1 TODO (Week-of-MAS work).....	58
23 Read Data Path (rd_data_path_fub).....	58
23.1 TODO (Week-of-MAS work).....	59
24 AXI4 Slave Protocol.....	59
24.1 Scope of This Section.....	59
24.2 Supported Features (v1).....	59
24.3 Optional Features (build-time).....	59
24.4 Unsupported Features.....	60
24.5 TODO.....	60
25 DFI v2.1 Master Protocol.....	60
25.1 Scope.....	60
25.2 Sub-Interfaces Supported.....	60
25.3 Sub-Interfaces Not Supported (v1).....	60
25.4 Phase / Gear Topology.....	61
25.5 TODO.....	61
26 APB CSR Slave Protocol.....	61

26.1 Protocol Compliance.....	61
26.2 Behavior.....	61
26.3 CDC.....	62
26.4 TODO.....	62
27 Register Map.....	62
27.1 Source of Truth.....	62
27.2 Staging vs. Live State.....	63
27.3 TODO.....	63
28 Runtime Overrides and Quiet Points.....	63
28.1 Override Categories.....	63
28.2 Quiet-Point Sequence.....	64
28.3 TODO.....	64
29 Family-Wide Config-Bit Applicability.....	64
29.1 Per-Bit Implementation Notes for DDR2 / LPDDR2.....	65
29.2 Soft-N/A Behavior Examples.....	65
29.3 TODO.....	65
30 Initialization Sequence.....	65
30.1 TODO.....	66
31 Refresh and Power-State Programming.....	66
31.1 TODO.....	66
32 Multi-Rank Programming.....	66
32.1 TODO.....	67
33 Error Handling.....	67
33.1 TODO.....	67

34 Build-Time Configuration Reference.....	68
34.1 TODO.....	68
35 Runtime Configuration Reference.....	68
35.1 TODO.....	68

List of Figures

No figures in this document.

List of Tables

No tables in this document.

1 Ddr2 Lpddr2 Mas Index

Generated: 2026-06-15

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

2 Document Information

2.1 DDR2 / LPDDR2 Family Controller Micro-Architecture Specification

Document Number: DDR2-LPDDR2-MAS-001 **Version:** 0.1 **Status:** Draft - Skeleton **Classification:** Open Source - Apache 2.0 License

2.2 Document Purpose

This Micro-Architecture Specification (MAS) is the implementation-level companion to the DDR2 / LPDDR2 Hardware Architecture Specification (HAS). Where the HAS answers “what is the architecture and why,” this MAS answers “what is in the RTL and how does it work.”

The MAS is the document RTL designers, verification engineers, and integrators consult when writing or reviewing SystemVerilog code, building testbenches, and stitching the controller into an SoC.

Target Audience:

- RTL designers implementing the FUBs described in this document
- Verification engineers writing per-FUB cocotb tests
- Integrators stitching the controller into an SoC
- Software engineers writing low-level memory configuration code (driver authors, bring-up engineers)

Companion Documents:

- DDR2/LPDDR2 Family Controller HAS (`../ddr2_lpddr2_has/`) — high-level architecture and design rationale

- DDR2/LPDDR2 DFI BFM documentation (in RTLDesignSherpa-DV) — verification-side reference
-

2.3 Document Scope

The MAS is the **micro-architecture** view: per-FUB inputs/outputs, internal state, FSM diagrams, datapath timing, register-level pipeline stages, and per-block timing budgets. It assumes the reader has read the HAS for context.

This document covers:

- Top-level integration wiring (ddr2_lpddr2_ctrl)
- Each FUB's interface signal list, internal storage, datapath flow, FSM, and timing budget
- AXI4 and DFI v2.1 wire-level protocol details specific to this controller
- APB programming interface and the full CSR register map
- Programming sequences (init, refresh, power-down, multi-rank, error handling)
- Build-time and runtime configuration reference

This document does **not** cover:

- Higher-level architectural rationale (see HAS)
 - Verification plans or coverage models (see dv/README.md)
 - Floorplan or layout guidance (project-specific)
 - Bit-level CSR YAML — that lives in regs/ and is the source of truth for CSR generation
-

2.4 Revision History

Version	Date	Author	Notes
0.1	2026-06-14	RTL Design Sherpa	Initial skeleton — chapter outline, FUB list, port list scaffold

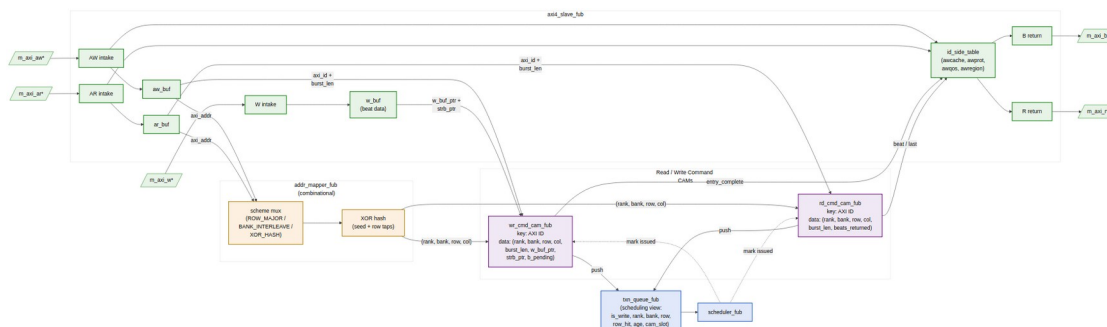
This chapter is the implementation-level architectural orientation: the top-level FUB stitching, the read/write datapath flow, and the clocking topology. Per-FUB detail is in §2.



The salient property: there is exactly **one** stage where AXI-layer concepts (burst, ID, write strobe) cross into DRAM-layer concepts (rank, bank, row, column). That stage is `addr_mapper` $\rightarrow$ `{rd,wr}_cmd_cam`. Upstream of `addr_mapper` everything is AXI; downstream of the CAMs everything is DRAM.

3.2 Read / Write Datapath: AXI → addr-hash → CAM

The user-facing entry point is `axi4_slave_fub`. From the slave, the read and write paths share `addr_mapper` (one cycle) and then split into two separate CAMs:



AXI → addr_mapper → CAMs Datapath

Source: [02_axi_to_cam_datapath.mmd](#)

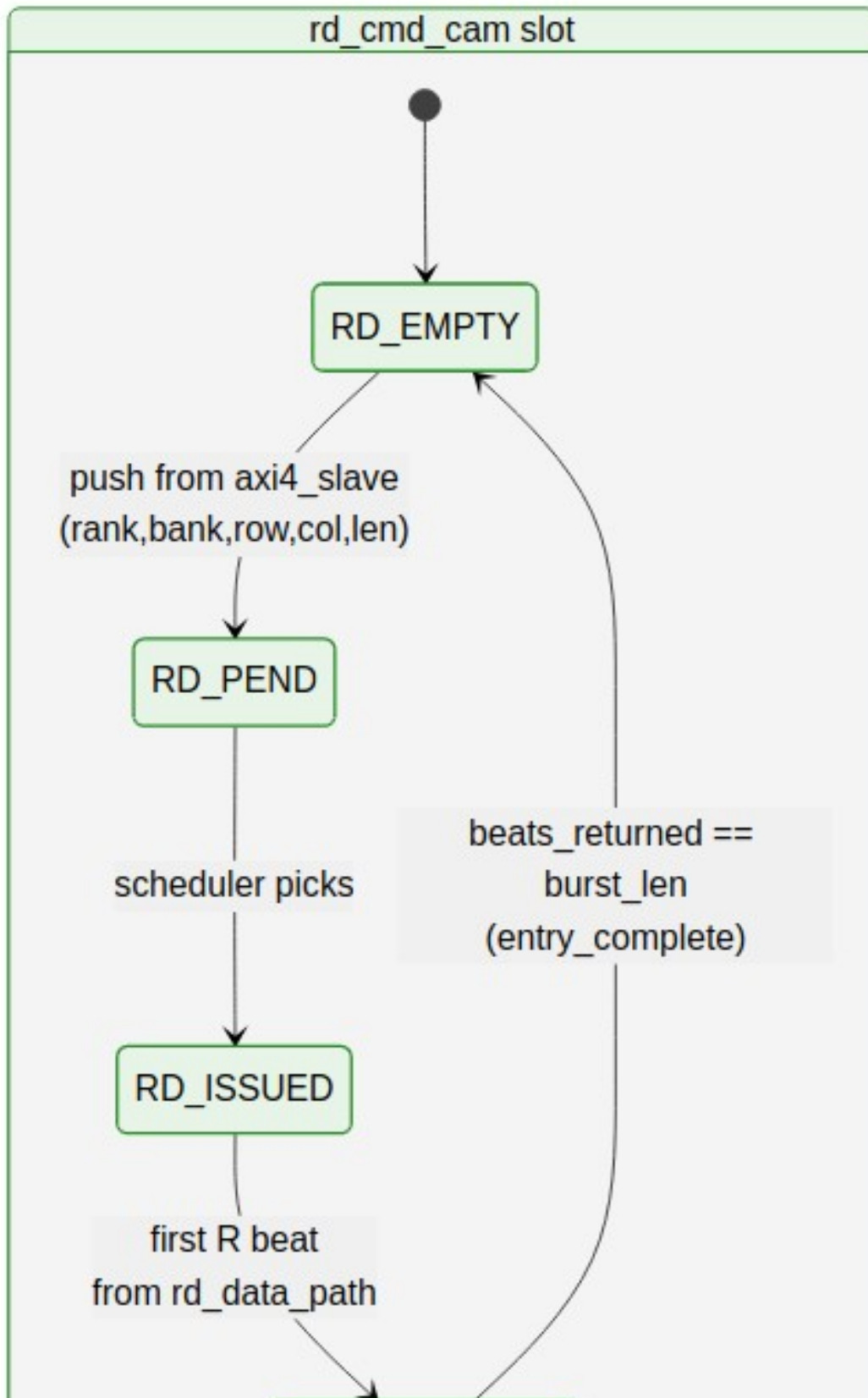
3.2.1 Why Two CAMs, Not One

The CAM split happens here, not at the scheduler, because read and write CAMs hold **different metadata** and have **different lifetimes**:

CAM	Key	Data	Cleared on
wr_cmd_cam	AXI ID	(rank, bank, row, col, burst_len, w_buf_ptr, strb_ptr)	B-channel response push
rd_cmd_cam	AXI ID	(rank, bank, row, col, burst_len, rid_counter, expected_beats)	last R beat of the burst

A single unified CAM would either carry both metadata sets (wasteful) or force an awkward “is_write” predicate on every lookup. Keeping the CAMs separate also means the read and write paths can be sized independently — the read path typically deeper because read latency is longer; the write path can be shallower because write completion is fire-and-forget.

The two CAMs have visibly different lifecycles — note in particular that the write side stays “alive” through the post-issue write window (waiting on `b_complete` from `xbank_timers`) while the read side just counts beats:



CAM Slot Lifecycles (read vs. write)

Source: [03_cam_lifecycles.mmd](#)

3.2.2 Why the XOR Happens Before the CAM

Address XOR / hash (`addr_mapper`) **must** happen before the CAM push because the CAM stores the post-decode (rank, bank, row, col) tuple — not the raw AXI address. Two reasons:

1. **Stable CAM key data.** The CAM is queried by the scheduler in the (rank, bank) dimension — for example, “do any pending writes target rank 0 bank 3, row R?” Storing the post-decode tuple makes this a direct comparison; storing the raw AXI address would force the scheduler to re-XOR on every match.
2. **Address-mapping scheme is per-controller, not per-burst.** The `ADDR_MAP_TUNING.scheme_or` field is global to the controller. If we stored raw addresses, a runtime scheme change (which can happen at any quiet point) would force a CAM walk to re-decode every entry. Storing the decoded tuple means a scheme change only affects **new** AXI bursts — old in-flight bursts continue with their decoded address. This matches the quiet-point semantics in HAS §6.3.

The `addr_mapper` is **combinational** — no pipeline stage between AXI intake and CAM push — so the timing budget for the XOR network is one cycle minus the AXI register-slice slack. See `ch02_blocks/03_addr_mapper.md` for the timing breakdown.

3.2.3 Why the AXI-side metadata stays in `axi4_slave_fub`

Some AXI fields (`awcache`, `awprot`, `awqos`, `awregion`, `bid/rid`) never need to cross into DRAM-layer state. These are held in small **side tables** inside `axi4_slave_fub`, indexed by the same AXI ID that keys the CAMs. The CAM holds the *DRAM* state; the side table holds the *AXI* state. On B/R completion, the CAM produces the completion signal and the side table produces the response metadata; the two are joined at the AXI return port.

This split keeps the CAM narrow (16 ID slots × ~36 bits in the default config) and pushes the AXI-specific clutter into the FUB that already owns the protocol.

3.3 Clocking Topology

Two external clocks; one CDC.

Clock	Frequency Range	Drives
<code>mc_clk</code>	100–500 MHz	All DRAM-layer FUBs: scheduler, bank machines, refresh, init, power, <code>cmd_encoder</code> , <code>gear_dfi</code> , <code>odt_ctrl</code> , data paths, and <code>axi4_slave</code>
<code>apb_pclk</code>	25–100 MHz	CSR slave only

Clock	Frequency Range	Drives
-------	-----------------	--------

`csr_apb_fub` owns the single CDC between `apb_pclk` and `mc_clk`. CSR overrides are handed off across this CDC into a quiet-point staging register; the actual override-application happens at the next quiet point on `mc_clk`. Quiet-point detection is in `power_state_fub` (no commands in flight, no refresh in progress).

There is **no CDC** in the AXI4 datapath — `axi4_slave_fub` runs in the `mc_clk` domain. If the SoC's AXI master is on a different clock, an external clock converter is required upstream of the controller.

3.4 DFI Phase / Gear Topology

The DFI v2.1 master interface is `N_PHASES`-deep. The MC clock drives one frame of `N_PHASES` DFI cycles per MC clock cycle. The `gear_dfi_fub` rewinds scheduler-issued commands and AXI data beats into the right per-phase slot per `WRPHASE` / `RDPHASE`. The scheduler does not see the phase dimension — its issue rate is one command per MC clock; the gear absorbs all of the phase-level scheduling.

The block diagrams and FSM diagrams referenced inline above are in `assets/mermaid/` (TBD; placeholders until the per-block detail in §2 is complete).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

4 Top-Level Port List

This chapter is the **wire-level** port list of the `ddr2_lpddr2_ctrl` top module — the exact signal names, directions, widths, and parameter dependencies that the integration script and SoC top-level use.

The implementation-level signal list is the same as the HAS overview view (HAS §2.4); this chapter restates it in the SystemVerilog-derivable form, with notes on bit-ordering, default tie-offs, and synthesis attributes where they matter.

The canonical machine-generated source is `regs/ddr2_lpddr2_ctrl_ports.svh` — what follows is the human-readable mirror.

4.1 Parameter-Dependent Widths (re-stated)

- $AW = AXI_ADDR_WIDTH$
- $DW = AXI_DATA_WIDTH$
- $SW = DW/8$
- $IDW = AXI_ID_WIDTH$
- $DFI_DW = DFI_DATA_WIDTH$
- $DFI_AW = DFI_ADDR_WIDTH$
- $NP = N_PHASES$
- $NR = NUM_RANKS$
- $NB = NUM_BANKS$
- $BA = \lceil \log_2(NUM_BANKS) \rceil$

4.2 AXI4 Slave Port List

See HAS §2.4 §1 for the table. The MAS adds nothing beyond what the HAS already published; the SystemVerilog declaration uses the same field names directly. AXI side-band signals (*awqos*, *awregion*, *arqos*, *arregion*) are observed but do not currently drive scheduler priority in v1; the wiring exists for the v2 QoS feature.

4.3 DFI v2.1 Master Port List

See HAS §2.4 §2. Additional MAS-level notes:

- The `dfi_cs_n[NR]` packed dimension is **outer-most** in SV declaration: `output [NP-1:0][NR-1:0] dfi_cs_n;`
- Per-rank `dfi_cke[NR]` is **outer-most**: `output [NR-1:0] dfi_cke;`
- The DDR2 *ras/cas/we* strobes are present even in LPDDR2 builds (driven all-1s, tied off by synthesis); the elaboration-time `MEMTYPE` parameter does *not* remove them from the port list, so the same SV header is usable for both flavors.

4.4 APB CSR Slave Port List

See HAS §2.4 §3. The APB clock and reset (*apb_pclk*, *apb_presetn*) are in the misc port group below; only the protocol signals are in this APB block.

4.5 Clocks, Resets, IRQs, Debug

See HAS §2.4 §4.

Additional MAS notes:

- `mc_rst_n` is asynchronous-assert, synchronous-deassert inside the controller (the deassert synchronizer lives in `power_state_fub`'s reset domain).
- `apb_presetn` is independently synchronized in `csr_apb_fub`.
- `irq_*` outputs are asserted in the `mc_clk` domain; the SoC's interrupt controller is responsible for re-synchronizing if it runs on a different clock.

4.6 SystemVerilog Header Snippet

The first 30 lines of the generated port header — the rest is per the tables above:

```
module ddr2_lpddr2_ctrl #(
  parameter string MEMTYPE           = "DDR2",
  parameter int   N_PHASES           = 4,
  parameter int   NUM_RANKS          = 1,
  parameter int   NUM_BANKS          = 8,
  parameter int   AXI_DATA_WIDTH     = 64,
  parameter int   AXI_ID_WIDTH       = 4,
  parameter int   AXI_ADDR_WIDTH     = 32,
  parameter int   ROW_WIDTH          = 14,
  parameter int   COL_WIDTH          = 10,
  parameter int   DFI_DATA_WIDTH     = 32,
  parameter int   DFI_ADDR_WIDTH     = 14,
  parameter int   WRPHASE             = 0,
  parameter int   RDPHASE            = 0,
  parameter int   TXN_QUEUE_DEPTH    = 16,
  parameter string SCHEDULER_MODE    = "000",
  parameter int   LOOKAHEAD_DEPTH_MAX = 4,
  parameter string PAGE_POLICY       = "HAPPY_HYBRID",
  parameter int   PAGE_PREDICTOR_TABLE_BITS = 12,
  parameter int   AGE_MAX             = 256,
  parameter int   REFRESH_DEFER_MAX  = 8,
  parameter string REFPB_POLICY      = "DARP",
  parameter string ODT_RULE_MULTIRANK = "JEDEC_DDR2",
  parameter int   SIM_INIT_SCALE     = 1
) (
  // Clocks and resets
  input logic mc_clk,
  input logic mc_rst_n,
  input logic apb_pclk,
  input logic apb_presetn,
```

```
// AXI4 slave, DFI master, APB slave, IRQs, debug – see chapter tables.
```

```
// ...  
);
```

The full module declaration is in `regs/ddr2_lpddr2_ctrl_ports.svh` (machine-generated).

5 Clocks and Reset

5.1 Clock Domains

Clock	Frequency	Polarity	Domain Members
mc_clk	100–500 MHz	Posedge	All DRAM-layer FUBs, axi4_slave, data paths
apb_pclk	25–100 MHz	Posedge	csr_apb_fub only

The DFI v2.1 interface is driven on mc_clk (the controller side of DFI is always controller-clock; the DFI PHY runs at the DRAM clock and the gear mapping is the gear FUB's responsibility).

5.2 Reset Topology

Reset	Polarity	Type	Domain
mc_rst_n	Active-low	Async assert, sync deassert	mc_clk
apb_presetn	Active-low	Async assert, sync deassert	apb_pclk

Each reset has its own 2-flop synchronizer for the deassert edge, instantiated in the FUB that owns the domain:

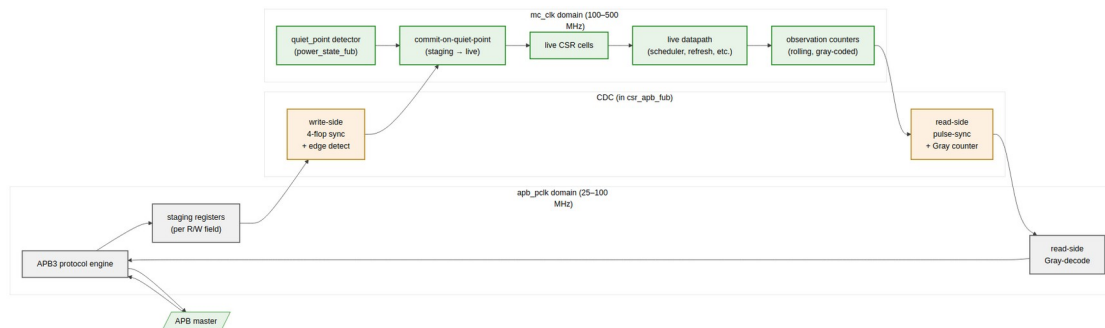
- mc_rst_n sync — in power_state_fub
- apb_presetn sync — in csr_apb_fub

Both synchronizers are clear-on-async-assert, hold-during-async-low. No reset-glitch filter is in the controller — the SoC's PMU is expected to drive a clean reset.

5.3 CDC

Exactly one CDC in the controller: APB → MC for CSR overrides and MC → APB for CSR readback. Both are handled in `csr_apb_fub`. The two directions use different mechanisms:

- **APB → MC** (write side): 4-flop synchronizer per register field, with a per-field `apb_write_strobe` edge-detect on the MC side to latch the new value into the staging register. Staging held until next quiet point.
- **MC → APB** (read side): per-counter pulse synchronizers feed Gray-coded saturating counters; APB reads sample the Gray code and decode locally. The MC-side counters never wrap synchronously between reads.



Clock Domains and CDC Topology

Source: [04_clocks_cdc.mmd](#)

5.4 Quiet Points

A **configuration quiet point** is the only safe moment to propagate CSR overrides from staging registers into the live datapath. Quiet points are defined by:

- `txn_queue_fub` is empty *or* all pending entries are in PENDING state (no ISSUED/COMPLETING)
- `refresh_mgr_fub` is in IDLE (not WAIT_BANK_GNTS, DO_REFRESH, DO_ZQCS)
- `power_state_fub` is in ACTIVE (not transitioning)
- No DFI command is in flight (the encoder's per-phase pipeline has drained)

When all four hold, `power_state_fub` asserts `quiet_point`. Override-staging in `csr_apb_fub` watches this signal and commits all queued overrides in one MC-clock edge.

Software triggers quiet-point drain via `CTRL.config_apply`. The CSR slave returns `STATUS.config_settled` when commit is complete.

5.5 Reset Sequence

On power-on:

1. `mc_rst_n` and `apb_presetn` are asserted (both low). PHY drives `dfi_init_complete = 0`.
2. SoC deasserts `apb_presetn`. `csr_apb_fub` comes out of reset; CSRs are R/W-able. Software loads timing CSRs (MR0..MR3, timings, address-map scheme, refresh tuning).
3. SoC deasserts `mc_rst_n`. All DRAM-layer FUBs come out of reset. Controller idles in **POST_RESET** state.
4. Software writes `CTRL.init_start = 1`. `init_engine_fub` walks the per-memtype step table, driving `dfi_reset_n`, MR loads, ZQCAL, etc.
5. On completion, `init_engine_fub` asserts `irq_init_done` (one cycle) and `STATUS.init_done = 1`. Controller transitions to **ACTIVE**.
6. AXI traffic from the host is honored.

The minimum gap between `apb_presetn` and `mc_rst_n` deassertion is 16 `mc_clk` cycles (the synchronizer chain depth). Software does not need to wait — the controller stalls at **POST_RESET** until both resets are released.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

6 Top-Level Integration (`ddr2_lpddr2_ctrl`)

Module: `ddr2_lpddr2_ctrl.sv` **Location:** `rtl/top/` **Category:** TOP (Structural only — no behavioral logic) **Status:** Skeleton

6.1 Purpose

`ddr2_lpddr2_ctrl` is the controller's top-level integration. It is **pure structural wiring** — every behavioral statement belongs in a FUB. The role of the top is:

1. Declare the external port list (see §1.2)
2. Instantiate every FUB
3. Wire FUB ↔ FUB interfaces
4. Fan-out per-rank / per-bank / per-phase signals where the FUB count is parametric

6.2 Instantiation Inventory

Instance Name	FUB	Count	Notes
u_axi4_slave	axi4_slave_fub	1	Single AXI4 slave port
u_addr_mapper	addr_mapper_fub	1	Combinational; no clock
u_rd_cmd_cam	rd_cmd_cam_fub	1	Read-side CAM
u_wr_cmd_cam	wr_cmd_cam_fub	1	Write-side CAM
u_txn_queue	txn_queue_fub	1	Unified pending queue
u_scheduler	scheduler_fub	1	
u_page_predictor	page_predictor_fub	0 or 1	Conditional on PAGE_POLICY == HAPPY_HYBRID
u_bank_machine[r][b]	bank_machine_fub	NUM_RANKS × NUM_BANKS	generate block, two-dimensional
u_xbank_timers	xbank_timers_fub	1	Internally per-rank tRRD/tFAW + global tCCD
u_refresh_mgr	refresh_mgr_fub	1	
u_init_engine	init_engine_fub	1	
u_power_state	power_state_fub	1	
u_cmd_encoder	cmd_encoder_fub	1	Parameterized by MEMTYPE
u_gear_dfi	gear_dfi_fub	1	
u_odt_ctrl	odt_ctrl_fub	1	
u_wr_data_path	wr_data_path_fub	1	
u_rd_data_path	rd_data_path_fub	1	

Instance Name	FUB	Count	Notes
u_csr_apb	csr_apb_fub	1	Only FUB in apb_pclk domain

6.3 Generate Blocks

The only parametric-fanout instances are the bank machines, fan-out arrays of per-rank CSRs, and per-rank ODT/CKE outputs:

```
generate
  for (genvar r = 0; r < NUM_RANKS; r++) begin : g_rank
    for (genvar b = 0; b < NUM_BANKS; b++) begin : g_bank
      bank_machine_fub #(.ROW_WIDTH(ROW_WIDTH)) u_bank_machine (
        .mc_clk          (mc_clk),
        .mc_rst_n        (mc_rst_n),
        .scheduler_if     (sched_to_bank[r][b]),
        .refresh_if       (refresh_to_bank[r][b]),
        .xbank_if         (xbank_to_bank[r][b]),
        .state_o          (bank_state[r][b]),
        .open_row_o       (bank_open_row[r][b]),
        .last_ref_age_o   (bank_last_ref_age[r][b])
      );
    end
  end
endgenerate
```

The for (genvar r) outer loop is the per-rank dimension; the inner is the per-bank dimension. The aggregation of bank_state[r][b] into the scheduler input is a structural concatenation only — no behavioral aggregation in the top.

6.4 What the Top Does Not Do

- No combinational logic beyond signal renaming or pure concatenation
- No CSR field expansion (the CSR slave handles its own field encoding)
- No clock gating (clock-gate insertion is a synthesis-script concern, not RTL)
- No reset synchronization (handled per-domain in power_state_fub and csr_apb_fub)
- No assign statements except for tying-off optional DFI signals (DDR2 vs LPDDR2 strobe ties)

This intentional poverty of the top file makes it the obvious place for the structural-only synthesis sanity check: “every line in this file is either an instantiation or a port-mapping; if a line is anything else, it is a bug.”

7 AXI4 Slave (axi4_slave_fub)

Module: axi4_slave_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Skeleton

7.1 Purpose

axi4_slave_fub is the AXI4 host-facing protocol engine. It accepts AW/W/AR transactions from the SoC, hands off the address to addr_mapper_fub for decode, and pushes the resulting transaction into the appropriate CAM (wr_cmd_cam_fub or rd_cmd_cam_fub). On B and R returns, it pulls completion signals from the CAMs and merges them with AXI-side metadata to produce protocol-compliant responses.

The FUB owns the AXI ID side table — the per-ID set of fields (awcache, awprot, awqos, awregion, bid echo, rid echo) that never need to cross into the DRAM-layer state.

7.2 External Interface (AXI side)

Per HAS §2.4 §1 — full AW/W/B/AR/R channel signal list. No additions in MAS.

7.3 Internal Buffers

Buffer	Depth	Width	Purpose
aw_buf	TXN_QUEUE_DEPTH/2 (default 8)	AW + IDW + 8(len) + 3(size) + 2(burst) + 4(cache) + 3(prot) + 4(qos) + 4(region)	Pending AW until CAM push
w_buf	TXN_QUEUE_DEPTH × max(burst_length)	DW + SW + 1(last) per beat	Write data staging
ar_buf	TXN_QUEUE_DEPTH/2 (default 8)	same AW fields	Pending AR until CAM push
b_response_fifo	small (4)	IDW + 2(resp)	Pending B responses

Buffer	Depth	Width	Purpose
r_resp onse_ fif o	small (4)	IDW + DW + 2(resp) + 1(last) per beat	Pending R responses
id_sid e_tabl e	2^IDW	4(cache) + 3(prot) + 4(qos) + 4(region)	Per-ID AXI metadata, indexed by awid/arid

The buffers above are FUB-internal; they do **not** appear in the architecture-level transaction queue (txn_queue_fub). The transaction queue holds the DRAM-layer view of pending work; these buffers hold the AXI-layer view of in-flight bursts.

7.4 Data Flow

7.4.1 Write Path

1. AW intake: capture aw* fields; push to aw_buf. Compute outstanding-count and assert awready if aw_buf not full.
2. W intake: accept w* beats into w_buf. Each W beat carries wstrb and wlast. The W path does not wait for AW — W can lead AW or trail it within AXI ordering rules.
3. CAM push: when both aw_buf head and the matching w_buf head are present (or when a write-only / address-only burst's prerequisites are satisfied), push to wr_cmd_cam_fub with:
 - key = awid
 - data = (rank, bank, row, col, burst_len, w_buf_ptr, strb_ptr)
 - where (rank, bank, row, col) come from addr_mapper_fub (combinational, same cycle)
4. Scheduler issue: when scheduler picks this entry, the FUB hands w_data_path_fub the w_buf_ptr and strb_ptr for it to walk the data beats.
5. B response: when wr_cmd_cam_fub reports the entry complete (last W beat issued + tDFI write to DFI), the FUB pops the AXI ID, joins with id_side_table[id].axi_metadata, and pushes b_response_fifo.
6. B emit: standard B-channel handshake; pop b_response_fifo.

7.4.2 Read Path

1. AR intake: capture ar* fields; push to ar_buf. Assert arready if not full.
2. CAM push: pop ar_buf head; push to rd_cmd_cam_fub with:
 - key = arid

- data = (rank, bank, row, col, burst_len, rid_counter, expected_beats)
 - where (rank, bank, row, col) come from addr_mapper_fub
 - rid_counter is the AXI ID echo
3. Scheduler issue: when scheduler picks this entry, it commands the issue of RD / RDA. rd_data_path_fub watches for the corresponding DFI rddata beats.
 4. R response: as each R beat completes, rd_data_path_fub strobes rd_cmd_cam_fub to decrement expected_beats; the CAM emits r_beat_to_axi with rid, beat data, rlast when the last beat arrives.
 5. R emit: standard R-channel handshake; pop r_response_fifo.

7.5 Outstanding Burst Tracking

The FUB enforces:

Limit	Constraint
Total in-flight reads	$\leq$ rd_cmd_cam_fub depth (default 16)
Total in-flight writes	$\leq$ wr_cmd_cam_fub depth (default 16)
Per-ID in-flight reads	$\leq$ MAX_PER_ID_READS parameter (default 4)
Per-ID in-flight writes	$\leq$ MAX_PER_ID_WRITES parameter (default 4)

If AXI_000_ACROSS_IDS == false, the per-ID counts reduce to “one in-flight per ID” — the AXI completion is strictly in-order per ID, but the issue order at the scheduler is still OoO.

7.6 Timing Budget

Path	Budget
awvalid $\rightarrow$ awready (combinational)	$\leq$ 0.7 of cycle
aw_buf push to addr_mapper valid	0 cycles (combinational)
addr_mapper valid to CAM push	1 cycle
wr_cmd_cam push to scheduler-visible	1 cycle
Worst path: awvalid $\rightarrow$ CAM-push register	2 cycles

The two-cycle AXI $\rightarrow$ CAM latency is the limit on AXI sustained issue rate; with default config it allows 1 burst per 2 cycles, well within the AXI burst rate at 200 MHz.

7.7 CSR Hooks

- `STATUS.axi_inflight_rd` (R only) — current count of in-flight read bursts
- `STATUS.axi_inflight_wr` (R only) — current count of in-flight write bursts
- `STATUS.axi_id_table_occ` (R only) — number of distinct AXI IDs currently in the side table
- `OBS_AXI_R_LATENCY_AVG`, `OBS_AXI_W_LATENCY_AVG` — driven from R/B-channel sampling in this FUB

7.8 TODO (week-of-MAS work)

- FSM diagram for the AW/W join and the AR intake (mermaid)
- B/R response-ordering corner cases (per HAS §3.1 OoO-across-IDs)
- Waveform examples (wavedrom) for back-pressure scenarios
- Detailed table of `id_side_table` field widths and reset values

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

8 Address XOR / Hash (`addr_mapper_fub`)

Module: `addr_mapper_fub.sv` **Location:** `rtl/fub/` **Category:** FUB (combinational; no clock)
Parent: `ddr2_lpddr2_ctrl` **Status:** Skeleton

8.1 Purpose

`addr_mapper_fub` decodes a flat `AXI_ADDR_WIDTH`-bit address into the DRAM-layer (rank, bank, row, column) tuple per the currently-active address-mapping scheme. The XOR / hash happens **here** — upstream of the read/write CAMs — so that the CAMs store the decoded tuple and not the raw AXI address.

The FUB is **pure combinational** — there is no clock, no flop, no FSM. The output is valid the same cycle as the input. The path is one of the timing-critical signatures in the design and is treated as a multi-cycle path only if synthesis cannot meet timing.

8.2 Mirror to the Python AddressMapping Class

This RTL is the mirror of the AddressMapping Python class in the DV repository (CocoTBFramework.components.dfi.address_mapping). Both produce **bit-for-bit identical** decoded outputs for the same input. The mirror is enforced by:

- Same scheme list (ROW_MAJOR, BANK_INTERLEAVE, XOR_HASH)
- Same field-bit positions per scheme
- Same XOR_HASH seed and tap matrix

The shared decode function eliminates a common class of bugs where simulation passes (because the BFM decodes one way) but RTL fails (because the controller decodes a different way).

8.3 Interface

Signal	Direction	Width	Description
axi_addr_i	input	AW	Flat AXI byte address
scheme_active_i	input	2	Runtime scheme select from ADDR_MAP_TUNING.scheme_or
xor_seed_i	input	8	Runtime seed for XOR_HASH (from CSR)
bg_field_pos_i	input	3	Bank-group field bit position (unused in DDR2/LPDDR2; reserved)
rank_o	output	$\log_2(NR)$	Decoded rank
bank_o	output	BA	Decoded bank
row_o	output	ROW_WIDTH	Decoded row
col_o	output	COL_WIDTH	Decoded column (byte-aligned within burst)

8.4 Scheme Decode

8.4.1 Scheme 1: ROW_MAJOR

The simplest mapping; one contiguous row per rank/bank stride. Bit layout:

```
| ... reserved ... | rank | row [ROW_WIDTH-1:0] | bank [BA-1:0] | col  
[COL_WIDTH-1:0] | byte_in_dq |
```

The byte-in-DQ field is the controller's column-byte offset and is consumed by wr_data_path_fub for wstrb masking; it is **not** part of the decoded col_o (which is column-word aligned).

ROW_MAJOR is best for sequential streaming workloads where a long burst stays in one row.

8.4.2 Scheme 2: BANK_INTERLEAVE

```
| ... reserved ... | rank | row [ROW_WIDTH-1:0] | col_hi [...] | bank  
[BA-1:0] | col_lo [...] | byte_in_dq |
```

Bank bits are placed **between** column-low and column-high bits, so consecutive cache lines round-robin across banks. This is the recommended default for general-purpose workloads with mixed access patterns; it produces higher bank-parallelism in the scheduler.

8.4.3 Scheme 3: XOR_HASH

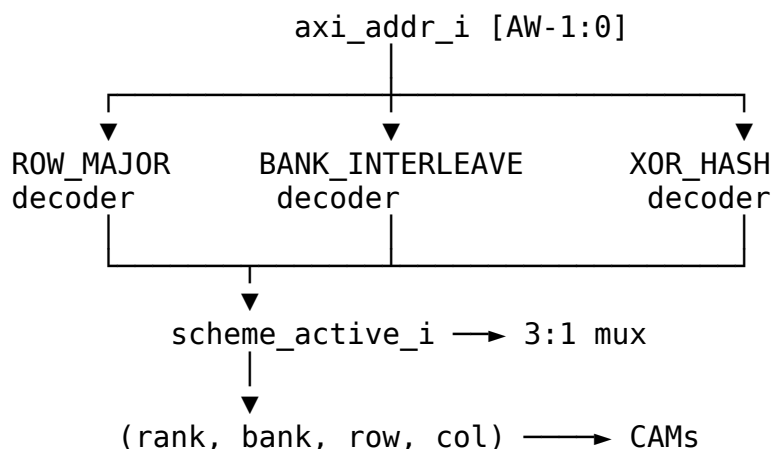
Same layout as BANK_INTERLEAVE, but the bank field and (a subset of) the column-high bits are XOR-hashed with row bits:

```
bank_hashed[i] = bank_raw[i] XOR row_raw[i] XOR row_raw[i + BA] XOR  
xor_seed[i]
```

The XOR_HASH scheme defeats pathological row-conflict patterns (e.g., a power-of-two stride that lands on the same bank in BANK_INTERLEAVE). The seed `xor_seed_i` is runtime-modifiable so a bring-up engineer can change the hash without rebuilding.

8.5 Runtime Scheme Mux

The output is multiplexed across the synthesized schemes per `scheme_active_i`. Only schemes in ADDR_MAP_SCHEMES_SYNTH are routed to the mux; un-synthesized schemes tie off and writing them via CSR returns `pslverr`.



8.6 Timing Budget

The decoder is purely combinational. The longest path in XOR_HASH is the XOR of (a) several address bits, (b) the seed, and (c) the row bits — about 5–6 levels of XOR gates plus a 3:1 mux. At 500 MHz this is comfortably within 0.5 ns of MC clock cycle time.

Path	Estimate (gate-level)
axi_addr_i → bank_o (ROW_MAJOR)	2 levels of MUX
axi_addr_i → bank_o (BANK_INTERLEAVE)	2 levels of MUX
axi_addr_i → bank_o (XOR_HASH)	5 levels XOR + 2 MUX
axi_addr_i → row_o	1 level MUX

If XOR_HASH does not meet timing at the target frequency, the FUB can be re-fitted with a single pipeline stage (the seed-XOR can absorb a flop without changing the per-tuple invariant — the seed only changes on quiet points anyway).

8.7 Verification

The RTL decoder and the Python AddressMapping class are co-validated by a small cocotb test that picks 10000 random addresses, asks each side to decode, and asserts bit-equality on the (rank, bank, row, col) tuple. This is the primary regression for addr_mapper_fub.

8.8 TODO

- Tap matrix for XOR_HASH — the exact bit mapping
- Synthesis hint for the XOR_HASH critical path
- Edge cases for $AW < (ROW_WIDTH + COL_WIDTH + BA + c\log_2(NR))$ (under-decoded address — should it tie-off or error?)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

9 Read Command CAM (rd_cmd_cam_fub)

Module: rd_cmd_cam_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Skeleton

9.1 Purpose

`rd_cmd_cam_fub` is the **read-side content-addressable storage** of in-flight read commands. It holds the DRAM-layer view (rank, bank, row, col) of every read burst that has been pushed by `axi4_slave_fub` and not yet fully returned on the AXI R-channel.

The CAM is keyed by **AXI ID**, allowing the scheduler to find all reads to a given (rank, bank) without iterating the queue and allowing `rd_data_path_fub` to find the right entry to decrement when a read beat returns.

The read CAM and write CAM are intentionally separate FUBs — see §1.1 for the rationale.

9.2 Parameters

Parameter	Default	Purpose
<code>RD_CAM_DEPTH</code>	16	Number of in-flight read slots
<code>IDW</code>	<code>AXI_ID_WIDTH</code>	Key width
<code>BURST_LEN_WIDTH</code>	8	Burst-length field width

9.3 Storage Per Slot

Field	Width	Description
<code>valid</code>	1	Slot occupied
<code>axi_id</code>	<code>IDW</code>	Key — the originating AXI ID
<code>rank</code>	$\$clog2(NR)$	Decoded rank
<code>bank</code>	<code>BA</code>	Decoded bank
<code>row</code>	<code>ROW_WIDTH</code>	Decoded row
<code>col_start</code>	<code>COL_WIDTH</code>	Starting column
<code>burst_len</code>	<code>BURST_LEN_WIDTH</code>	Total beats in burst
<code>beats_returned</code>	<code>BURST_LEN_WIDTH</code>	Beats already returned on R
<code>issued</code>	1	Scheduler has picked this entry
<code>last_beat_at</code>	small	Cycle of last beat return (for latency telemetry)

Total per-slot width (default config): ~46 bits.

Total storage (16 entries × 46 b): ~92 bytes — small enough to fit comfortably in distributed flops with no SRAM macro.

9.4 Interfaces

9.4.1 Push (from axi4_slave_fub)

Signal	Direction	Width	Description
push_valid	input	1	New read burst arriving
push_ready	output	1	CAM has a free slot
push_id	input	IDW	AXI ID
push_rank	input	$\$clog2(NR)$	from addr_mapper
push_bank	input	BA	
push_row	input	ROW_WIDTH	
push_col	input	COL_WIDTH	
push_len	input	BURST_LEN_WIDTH	

9.4.2 Scheduler Query

The scheduler queries the CAM to: - Find all unissued entries matching a target (rank, bank) — for next-issue selection - Find row-hit candidates (entries whose row == open_row[rank][bank])

Signal	Direction	Description
q_rank_i	input	Query rank
q_bank_i	input	Query bank
q_row_i	input	Query row (for row-hit check)
match_unissued_o	output	One-hot vector of slots matching (rank, bank) and not issued
match_rowhit_o	output	One-hot vector of slots matching (rank, bank, row) and not issued
match_data_o	output	The selected slot's payload (rank, bank, row, col, len)

9.4.3 Issue Notification

When the scheduler picks an entry from this CAM:

Signal	Direction	Description
issued_slot_i	input	Slot index to mark as issued
issued_we_i	input	Mark-issued strobe

9.4.4 Beat Return (from rd_data_path_fub)

Signal	Direction	Description
beat_id_i	input	AXI ID of returning beat
beat_we_i	input	Beat-arrived strobe
entry_complete_o	output	One-hot vector of slots that just completed (last beat of their burst)

On entry-complete the slot's valid is cleared in the next cycle.

9.5 Lookup Mechanism

The CAM uses a **parallel match-line** structure: each slot has a small comparator for {rank, bank} that drives a per-slot match wire. The scheduler reads the full match-vector and picks among matches using a priority encoder (age-priority — oldest slot wins ties).

For the typical RD_CAM_DEPTH of 16 with BA=3, $\lceil \log_2(NR) \rceil = 1..2$ (1 for single-rank, 2 for quad-rank), this is 16 4-5-bit equality comparators in parallel — trivial silicon.

9.6 Per-ID Beat Counting

Because reads return in-order *per AXI ID* (an AXI ordering rule), the read CAM does not need a per-beat slot table. A slot's beats_returned increments on each beat that returns with that ID, and when beats_returned == burst_len the slot completes and clears.

If AXI_000_ACROSS_IDS == false (rare), the SoC has asked for in-order return across all IDs, and the read CAM enforces this by issuing a head-of-queue priority hint to the scheduler — entries are still cleared in arrival order. The OoO-across-IDs default is true; per-ID in-order is the AXI default and is preserved.

9.7 Timing Budget

Path	Budget
push_valid → slot occupy (1 cycle)	1 cycle
q_rank_i / q_bank_i → match*_o	combinational
beat_id_i →	1 cycle

Path	Budget
beats_returned[slot]+1	
entry_complete_o → slot clear	1 cycle

The combinational match path is on the scheduler's critical loop and is shared with `wr_cmd_cam_fub` and the bank machines' state matrix. See `ch02_blocks/07_scheduler.md` for the full critical-path breakdown.

9.8 CSR Hooks

- `STATUS.rd_cam_occ` — current occupancy
- `OBS_RD_CAM_HIGH_WATER` — peak occupancy observed since last read

9.9 TODO

- Mermaid for the parallel match-line topology
- Wavedrom: push → scheduler-match → issue → beats return → clear
- Per-ID head-of-queue ordering corner case spec

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

10 Write Command CAM (`wr_cmd_cam_fub`)

Module: `wr_cmd_cam_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl`
Status: Skeleton

10.1 Purpose

`wr_cmd_cam_fub` is the **write-side content-addressable storage** of in-flight write commands. It mirrors the read CAM's role for write traffic but carries different per-slot metadata:

- Pointer into the AXI-side write-data buffer (`w_buf_ptr`)
- Pointer into the AXI-side write-strobe buffer (`strb_ptr`)
- Per-slot beats-issued counter (so the FUB knows when the last W beat has been pushed to DFI write data)

Separation from the read CAM lets the write path sit shallow — writes complete on B response, which is a single AXI event, so the write CAM does not need to track per-beat return.

10.2 Parameters

Parameter	Default	Purpose
WR_CAM_DEPTH	16	Number of in-flight write slots
IDW	AXI_ID_WIDTH	Key width
W_BUF_PTR_WIDTH	$\$clog2(W_BUF_DEPTH)$	Pointer into AXI W buffer
BURST_LEN_WIDTH	8	Burst-length field width

10.3 Storage Per Slot

Field	Width	Description
valid	1	Slot occupied
axi_id	IDW	Key — the originating AXI ID
rank	$\$clog2(NR)$	Decoded rank
bank	BA	Decoded bank
row	ROW_WIDTH	Decoded row
col_start	COL_WIDTH	Starting column
burst_len	BURST_LEN_WIDTH	Total beats
beats_issued	BURST_LEN_WIDTH	Beats already pushed to DFI write data
w_buf_ptr	W_BUF_PTR_WIDTH	Pointer into the AXI W-buffer for next beat
strb_ptr	W_BUF_PTR_WIDTH	Pointer into the per-beat strobe buffer
issued	1	Scheduler has picked this entry
b_pending	1	Awaiting B response from DFI write completion

Total per-slot width: ~58 bits in the default config. Total storage (16×58): ~116 bytes, distributed flops.

10.4 Interfaces

10.4.1 Push (from axi4_slave_fub)

Same shape as the read CAM push, plus `w_buf_ptr` and `strb_ptr` (which the AXI slave allocates at push time).

10.4.2 Scheduler Query

Same shape as the read CAM. The scheduler matches by (rank, bank) and selects with row-hit priority.

10.4.3 Issue Notification

When the scheduler picks a write, the CAM: 1. Marks `issued = 1` 2. Begins handing per-beat data to `wr_data_path_fub` via the `beat_data_pull` interface (below)

10.4.4 Beat-Pull (to wr_data_path_fub)

Signal	Direction	Description
<code>beat_pull_o</code>	output	Strobe — pull next beat for this slot
<code>beat_pull_slot_o</code>	output	Slot index whose beat is being pulled
<code>beat_pull_ptr_o</code>	output	<code>w_buf_ptr + beats_issued</code>
<code>beat_pull_strb_o</code>	output	<code>strb_ptr + beats_issued</code>
<code>beat_pull_last_o</code>	output	1 when this is the burst's final beat

`wr_data_path_fub` walks the AXI W buffer at the supplied pointer and pushes to DFI `wrdata`. On the last beat, it asserts `b_pending_set` to the CAM.

10.4.5 Completion (from DFI write window expiration)

Signal	Direction	Description
<code>b_pending_set_i</code>	input	Last W beat pushed; await <code>tCWL+tWR</code> window
<code>b_complete_i</code>	input	Write window expired (from <code>xbank_timers</code>)
<code>entry_complete_o</code>	output	Burst is fully complete; signal <code>axi4_slave_fub</code> to push B response

10.5 Difference From Read CAM

Aspect	Read CAM	Write CAM
Lifecycle	Push → issue → beats return → clear	Push → issue → beats issued → write window expires → B →

Aspect	Read CAM	Write CAM
		clear
Beat tracking	Counts beats <i>returned</i> from DFI	Counts beats <i>issued</i> to DFI; also tracks the post-issue write window
AXI ID echo	rid returned with each R beat	bid returned once at B-response time
Pointer fields	none (just a beat counter)	w_buf_ptr, strb_ptr into AXI-side buffers
Completion driver	rd_data_path (last beat of R)	xbank_timers (write window expiration) → wr_data_path

The asymmetry justifies the separate FUBs. Trying to share a CAM would either inflate the read slot to carry write-only fields, or vice versa.

10.6 Timing Budget

Same shape as the read CAM. The combinational match-line path is shared with rd_cmd_cam_fub in the scheduler's match aggregation.

10.7 CSR Hooks

- STATUS.wr_cam_occ — current occupancy
- STATUS.wr_b_pending — count of slots awaiting B response
- OBS_WR_CAM_HIGH_WATER — peak occupancy

10.8 TODO

- Mermaid for the issued → beat-pull → b_pending → b_complete lifecycle
- Wavedrom showing a multi-beat write burst from CAM push to B response
- Backpressure path when wr_data_path can't keep up with beat_pull (rare; DFI wrdata is wider than AXI W)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

11 Transaction Queue (txn_queue_fub)

Module: txn_queue_fub.sv **Location:** rtl/fub/ **Status:** Skeleton

See HAS §3.2 txn_queue for the architectural view. This MAS block is the implementation-level detail.

11.1 Relationship to the CAMs

The txn_queue is the **unified scheduling view**. Each pending CAM slot (read or write) is reflected as one queue entry, but the queue's entry only carries fields the scheduler needs:

{is_write, rank, bank, row, row_hit_cached, age, state, cam_slot_idx}

The reverse pointer cam_slot_idx links back to the corresponding rd_cmd_cam or wr_cmd_cam entry. The queue's job is purely scheduling-policy storage; the CAMs carry the AXI-side and beat-tracking detail.

11.2 TODO

- Entry-format width breakdown
- Age-saturation specifics
- row_hit_cached update path (per HAS §3.2 — cached at insert; updated on row-state change)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

12 FR-FCFS Scheduler (scheduler_fub)

Module: scheduler_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2_lpddr2_ctrl
Status: Draft v0.1

Architectural context: HAS §3.2 (scheduler, priority function, lookahead, force_inorder, refresh-priority). This block-level MAS section is the implementation view: signals, internal datapath, pipeline staging, critical paths, and verification scenarios.

12.1 Purpose

scheduler_fub is the central command issue engine. Every MC clock cycle it:

1. Snapshots txn_queue and all bank_machine states
2. Computes a per-entry readiness vector (which queue entries *could* issue right now)
3. Applies priority masking (init, refresh, force-inorder, row-hit-first, age tie-break)
4. Picks at most one winner via parallel priority encoders
5. Generates the DRAM command (issue_op, operands) and the CAM “mark issued” strobe

Issue rate is **one command per MC clock**. Multi-cycle commands (LPDDR2's 2-cycle CA encodings) are absorbed inside cmd_encoder_fub and gear_dfi_fub; the scheduler still sees one issue per cycle.

The block is the **single hardest synthesis-timing FUB** in the design — the parallel match-line into per-entry readiness, then into priority encoders, is what sets the MC clock frequency ceiling. Everything else has slack.

12.2 Synthesis Parameters

Parameter	Source	Effect on this FUB
SCHEDULER_MODE	top	"000" synthesizes the full FR-FCFS comparator network; "INORDER" synthesizes only the first-ready FIFO path (smaller area, no row-hit reordering).
TXN_QUEUE_DEPTH	top (default 16)	Number of parallel readiness comparators
LOOKAHEAD_DEPTH_MAX	top (default 4)	Width of the lookahead window peek into txn_queue
PAGE_POLICY	top	Picks which Stage-4 auto-precharge decoder is synthesized
NUM_RANKS, NUM_BANKS	top	Width of the bank-state input matrix
AGE_MAX	top (default 256)	Width of per-entry age counter

Parameter	Source	Effect on this FUB
		(clog2(AGE_MAX))

The "INORDER" synthesis variant is a different decoder tree, not a runtime gate of the OoO tree. The runtime SCHED_TUNING.force_inorder bit is honored *only* when SCHEDULER_MODE == "000" was synthesized (the bit becomes a tied observation in INORDER builds, per HAS §3.2).

12.3 Interface

12.3.1 Inputs

Signal	Direction	Width	Description
mc_clk, mc_rst_n	input	1, 1	MC clock + async-assert / sync-deassert reset
q_entries_i[TXN_QUEUE_DEPTH-1:0]	input	per HAS §3.2	Live queue snapshot: per entry {valid, is_write, rank, bank, row, col, burst_len, row_hit_cached, age, state, cam_slot_idx}
bank_state_i[NUM_RANKS][NUM_BANKS]	input	per bank_machine_fub	Per-(rank, bank): state, open_row, accepts_act/rd/wr/pre/ref
xbank_blocks_i	input	struct	blocks_act_per_rank[NR], blocks_xrank_rd, blocks_xrank_wr
refresh_wants_i	input	1	One-bit “refresh due now” from refresh_mgr_fub
refresh_done_for_i[NUM_RANKS][NUM_BANKS]	input	NR×NB	Per-(rank, bank) “refresh handshake granted” from refresh_mgr_fub
init_in_progress_i	input	1	From init_engine_fub — blocks all normal traffic
predict_hit_i[NUM_RANKS][NUM_BANKS]	input	NR×NB	HAPPY predictor output (tied 0 when PAGE_POLICY != HAPPY_HYBRID)
cfg_force_inorder_i	input	1	SCHED_TUNING.force_inorder

Signal	Direction	Width	Description
	ut		runtime override
cfg_lookahead_active_i	inp ut	4	SCHED_TUNING.lookahead_active
cfg_page_policy_or_i	inp ut	2	REFRESH_TUNING.page_policy_or
cfg_happy_enable_i	inp ut	1	SCHED_TUNING.happy_enable

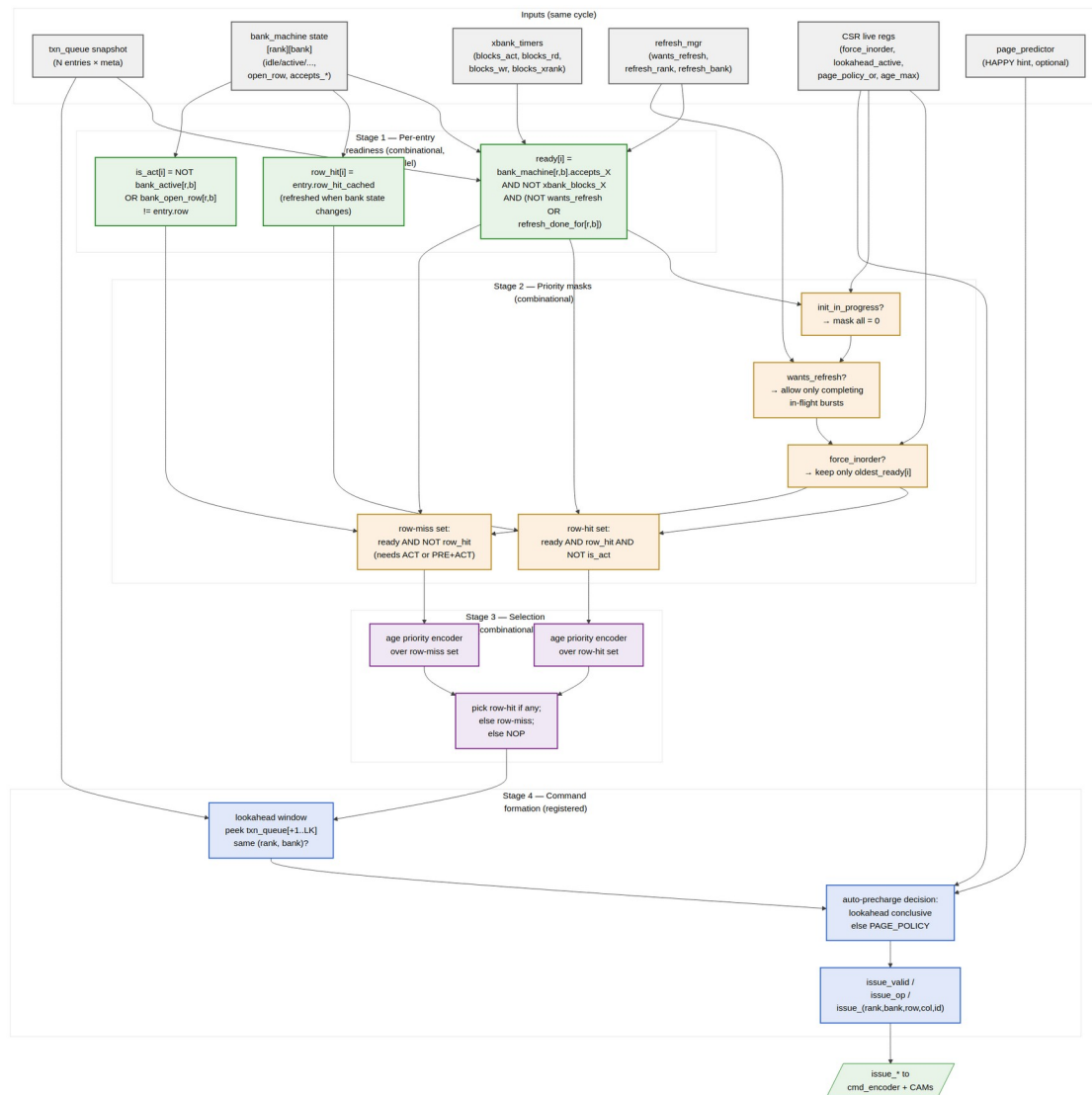
12.3.2 Outputs

Signal	Direction	Width	Description
issue_valid_o	outp ut	1	A command is being issued this cycle
issue_op_o	outp ut	4	One of {NOP, ACT, RD, RDA, WR, WRA, PRE, PREA, REF, REFPB, MRS, ZQCS, ZQCL}
issue_rank_o	outp ut	$\text{clog}_2(\text{NR})$	Target rank
issue_bank_o	outp ut	$\text{clog}_2(\text{NB})$	Target bank
issue_row_o	outp ut	ROW_WIDTH	Target row (for ACT) or open-row pass-through (for RD/WR)
issue_col_o	outp ut	COL_WIDTH	Target column (for RD/WR)
issue_axi_id_o	outp ut	AXI_ID_WIDTH	Originating AXI ID (passed to the CAM for completion routing)
issue_cam_slot_o	outp ut	$\text{clog}_2(\text{CAM_DEPTH})$	Index into rd_cmd_cam or wr_cmd_cam to mark “issued”
issue_is_write_o	outp ut	1	Picks wr_cmd_cam vs rd_cmd_cam for the mark-issued strobe
predict_query_o	outp ut	struct	(rank, bank, row) query to page_predictor_fub for stage-4 auto-pre
dbg_inorder	outp	1	(debug) High when current cycle picked the

Signal	Direction	Width	Description
_active_o	ut		in-order path
dbg_refresh_blocking_o	output	1	(debug) High when wants_refresh reduced the candidate pool

12.4 Decision Pipeline

The decision is a single MC-clock-cycle combinational pipeline broken into four conceptual stages, finishing in a registered output. Stages 1–3 are combinational; Stage 4 latches the final command into output flops.



Scheduler Priority Pipeline

Source: [05_scheduler_priority_pipeline.mmd](#)

12.4.1 Stage 1 — Per-Entry Readiness (combinational, N parallel comparators)

For each of `TXN_QUEUE_DEPTH` entries, in parallel:

```
ready[i] = q_entries[i].valid
        AND (q_entries[i].state == PENDING)
        AND bank_machine[r,b].accepts_X
        AND NOT xbank_blocks_X
        AND ( NOT refresh_wants
              OR bank_machine[r,b].in_refresh_done_set )
```

```
is_row_hit[i] = q_entries[i].row_hit_cached
                ( cached at queue insert; refreshed when
bank_machine[r,b]
                state changes – see "Cache Coherency" below )
```

```
needs_act[i] = NOT bank_active[r,b]
                OR bank_open_row[r,b] != q_entries[i].row
```

Where X is rd if $is_write == 0$, wr if $is_write == 1$. $r = q_entries[i].rank$, $b = q_entries[i].bank$.

$ready[i]$ answers “could I issue *some* command for this entry this cycle without violating any per-bank or cross-bank timer?” $is_row_hit[i]$ answers “if I do, is it a column command on the already-open row?” $needs_act[i]$ answers “is the preliminary command actually ACT (or PRE+ACT) rather than RD/WR?”

12.4.2 Stage 2 — Priority Masking (combinational)

Five masks, computed in parallel from the Stage-1 vector and the config inputs:

Mask	Definition	Effect
$init_mask_k$	$init_in_progress ? 0 : ready[i]$	Block all normal entries during init
$refresh_mask$	$refresh_wants ? (ready[i] \text{ AND } bank_in_refresh_done_set) : ready[i]$	Allow only entries whose target rank+bank has already been granted the refresh handshake (i.e., already transitioned out of REFRESHING, or never entered it)
$inorder_mask$	$force_inorder ? oldest_ready_one_hot : ready[i]$	Collapse to first-ready FIFO
row_hit_mask	$ready[i] \text{ AND } is_row_hit[i] \text{ AND } NOT\ needs_act[i]$	Row-hit candidates (column-only)
row_miss_mask	$ready[i] \text{ AND } (NOT\ is_row_hit[i] \text{ OR } needs_act[i])$	Row-miss candidates (need ACT or PRE+ACT)

The composed candidate vectors are:

```
cands_row_hit = init_mask AND refresh_mask AND inorder_mask AND
row_hit_mask
cands_row_miss = init_mask AND refresh_mask AND inorder_mask AND
row_miss_mask
```

12.4.3 Stage 3 — Selection (combinational priority encoders)

Two parallel age-priority encoders, then a 2:1 mux:

```
winner_hit  = age_pe(cands_row_hit)    // index of oldest entry in
cands_row_hit, or none
winner_miss = age_pe(cands_row_miss)  // index of oldest entry in
cands_row_miss, or none

picked = winner_hit  IS_VALID ? winner_hit
        : winner_miss IS_VALID ? winner_miss
        : NONE      // issue NOP
```

The age priority encoder is the slowest path in this stage — it's an N-entry tournament on $\text{clog2}(\text{AGE_MAX}) = 8$ -bit ages. For $N = 16$ (the default), that's 4 levels of pairwise compare-and-select. Synthesizable in two LUT layers per level on 7-series; ~30 LUT-delays in the typical config.

12.4.4 Stage 4 — Command Formation (registered)

Once an entry is picked, Stage 4 turns it into a DRAM command. This stage **may pre-flop intermediate signals** for timing closure on the highest clock targets (see “Pipeline Staging” below).

1. **Lookahead peek** — examine the next `cfg_lookahead_active` queue entries *after* the picked one to see if any target the same (rank, bank).
2. **Auto-precharge decision** — per HAS §3.2 (lookahead first, fallback to `PAGE_POLICY`):
 - Same-bank entry found AND its row matches the open row → **keep open** → bare RD / WR
 - Same-bank entry found AND its row differs → **close** → RDA / WRA
 - No same-bank entry within window → consult `cfg_page_policy_or`:
 - OPEN → bare RD / WR
 - CLOSE → RDA / WRA
 - HAPPY_HYBRID → query `predict_hit_i[r][b]`; if predicted hit → bare; else RDA/WRA
3. **Op encoding** — generate the 4-bit `issue_op_o` from (`is_write`, `needs_act`, `auto_pre`):
 - `needs_act` and bank state == IDLE → ACT
 - `needs_act` and bank state == ACTIVE (different row) → PRE (the column command follows next cycle after tRP)
 - NOT `needs_act` and `is_write` → WR or WRA
 - NOT `needs_act` and NOT `is_write` → RD or RDA

4. **Latch** outputs into the issue flops.

12.4.5 Refresh Window Behavior

When `refresh_wants_i` is high, `refresh_mgr_fub` is asserting `refresh_req` to one or more bank machines. Those bank machines drive `refresh_done_for_i[r][b] = 1` only after reaching IDLE and handing over the grant. While `refresh_done_for_i[r][b] == 0`, that bank's entries are masked out — no new column commands to a bank that's about to refresh.

Bank machines for *other* ranks (in the multi-rank REFab round-robin) or *other* banks (in REFpb) remain available, so the scheduler continues to drain those during the refresh window. This is what makes per-rank REFab dispatch beneficial — the non-refreshing ranks keep their throughput.

When all required refreshes complete and `refresh_wants_i` drops, the masks lift and normal FR-FCFS resumes the next cycle.

12.5 Cache Coherency: `row_hit_cached`

`q_entries[i].row_hit_cached` is computed at queue insertion (by `axi4_slave_fub` via a one-cycle query to `bank_machine[r,b].open_row`) and stored in the queue entry. This avoids re-querying every cycle. But the bank state can change between insertion and selection:

- When `bank_machine[r,b]` transitions ACTIVE → ACTIVE on a different row (via PRE + ACT to a new row), all `q_entries` with `(rank, bank) == (r, b)` need their `row_hit_cached` recomputed against the new `open_row`.
- When `bank_machine[r,b]` transitions IDLE → ACTIVATING, the queue entries get `row_hit_cached = 1` for entries whose row matches the in-flight ACT.

This is implemented as a **broadcast update**: on every bank state transition that affects the open row, the bank machine emits `open_row_changed[r][b]` + the new `open_row`. The queue has a per-entry update path that, in parallel, recomputes `row_hit_cached`. This is a one-cycle update — the recomputed value is visible to the scheduler in the cycle *after* the bank transition.

The one-cycle lag means the scheduler may make an “outdated row-hit” decision once per row change. The downside is one mis-categorized RD/WR (issued as bare when RDA was warranted, or vice versa); the worst case is one extra PRE later. This is a deliberate trade — the alternative is a combinational broadcast from bank state through every queue entry's row-hit comparator, which would inflate the Stage-1 critical path.

12.6 Pipeline Staging

The default build is a **single-cycle combinational** scheduler — Stages 1–3 are combinational; Stage 4's auto-precharge decoder is combinational; only the final issue-flop is registered. On 7-series FPGA at 200 MHz this closes timing comfortably. For 14 nm targets at 500+ MHz, two timing-closure escape hatches:

1. **Stage 3 → Stage 4 flop insertion** — register picked between stages. Adds one cycle of issue latency. Throughput is unchanged because the pipeline is single-issue anyway; the only visible difference is that a request inserted at cycle T can first issue at cycle T+3 instead of T+2.
2. **Stage 1 partition by bank** — partition the readiness comparator network by (rank, bank) so each partition produces a per-bank ready vector independently, then aggregate. Useful when $\text{NUM_RANKS} \times \text{NUM_BANKS}$ is small (the default $1 \times 8 = 8$ partitions) and routing congestion dominates.

These are not on by default; they're synthesis-script switches, not RTL parameters.

12.7 Strict In-Order Mode (collapse)

When `cfg_force_inorder_i = 1` (and `SCHEDULER_MODE == "000"` is synthesized):

- `inorder_mask` collapses to `oldest_ready_one_hot` — a one-hot vector that picks the *oldest valid ready entry* by age.
- This drops the row-hit prioritization (no row-hit vs row-miss split — both go through `cands_row_miss` since age order rules).
- Refresh priority is preserved (JEDEC requirement).
- Lookahead and HAPPY remain on at elaboration but can be independently disabled at runtime (`cfg_lookahead_active = 0`, `cfg_happy_enable = 0`) for a pure first-ready FIFO.

When `SCHEDULER_MODE == "INORDER"` is synthesized:

- The Stage-2 / Stage-3 priority encoders are different RTL: the row-hit/row-miss split is omitted; only the oldest-ready selector is built.
 - `cfg_force_inorder_i` becomes a tied observation bit at `STATUS.force_inorder_obs = 1`.
-

12.8 Critical-Path Analysis (rough budget at 500 MHz target, 2 ns cycle)

Path	Levels (approx)	Budget
bank_state_i.accepts_X → ready[i] → row-hit mask → cands_row_hit[i]	4 LUT levels	0.7 ns
cands_row_hit → age priority encoder → winner_hit (16-entry tournament)	4 levels × 2 LUT each = 8	1.2 ns
winner_hit / winner_miss → 2:1 mux → picked	1 LUT level	0.2 ns
Routing / setup margin		0.3 ns
Total Stage 1–3		2.4 ns

So at 500 MHz the path **misses by ~400 ps** and needs Stage 3 → Stage 4 flop insertion (the first escape hatch above). At 200 MHz target (5 ns cycle, embedded SoC default), the path closes with ~2.6 ns of slack — no escape hatch needed.

The 500 MHz analysis is FPGA-pessimistic; ASIC targets close 500 MHz on this path comfortably with standard pipelining.

12.9 CSR Hooks

The scheduler drives several CSR observability fields:

CSR field	Source signal	Use case
STATUS.issue_op_obs (R)	Last issued issue_op_o	Bring-up debug: what was the last DRAM command
STATUS.scheduler_idle_pct (R)	Rolling counter — fraction of cycles issue_valid_o = 0	Bandwidth-headroom telemetry
OBS_ROW_HIT_RANK<R>_BANK<N> (R)	Incremented when row-hit path wins for that bank	Per-bank row-hit rate (HAS §6.3)
OBS_INORDER_FALLBACKS (R)	Incremented when force_inorder ate a row-hit	Tells software how much OoO is currently giving up

CSR field	Source signal	Use case
	win	
OBS_LOOKAHEAD_HI TS (R)	Incremented when Stage-4 lookahead was conclusive	Tunes LOOKAHEAD_DEPTH_MAX characterization sweep
STATUS.force_ino rder_obs (R)	Echo of effective force- inorder state	Software-readable confirmation

12.10 Verification Notes (cocotb test plan)

Scenario	What it proves
Single-write, single-bank, single-rank	Smoke test: ACT → WR → b_complete cycle
Single-read, single-bank, single-rank	Smoke test: ACT → RD → beat return
Two reads, same bank, same row, oldest first	Row-hit reordering not invoked (already in order)
Two reads, same bank, <i>different</i> rows, oldest is row-conflict	OoO: younger row-hit wins over older row-miss
Same as above with force_inorder = 1	OoO disabled: older row-miss wins despite younger row-hit
Cross-bank: rd to bank 0 and rd to bank 1, bank 0 row-miss, bank 1 row-hit	Bank parallelism: bank-1 issue fires first
Cross-rank (NUM_RANKS=2): rd to rank 0 bank 3 and rd to rank 1 bank 3	Per-rank bank parallelism
Refresh during in-flight burst	Refresh window correctly drains and resumes
Refresh during multi-rank traffic (REFab on rank 0, traffic on rank 1)	Per-rank REFab dispatch: rank-1 traffic continues
Age-saturation under sustained row- conflict from younger IDs	Older row-conflict eventually wins via age priority
LOOKAHEAD_DEPTH_MAX = 4, lookahead concludes auto-precharge correctly	Lookahead path
LOOKAHEAD_DEPTH_MAX = 0, HAPPY	Fallback path through page

Scenario	What it proves
predictor used	predictor
PAGE_POLICY = CLOSE, every column command is auto-pre	CLOSE policy
Init-in-progress blocks normal traffic	init_mask gating
Queue full + back-pressure to AXI	axi4_slave does not accept new bursts

12.11 Open Questions / Future Work

- Should QoS (awqos / arqos) be promoted from the v1 “no behavior” pass-through to a real priority boost in the FR-FCFS function? Currently HAS §3.2 leaves QoS as a side-band hint for v2. The hook would be a qos_boost_mask between the row-hit and row-miss stages.
- The cross-rank read-to-write turnaround (tRTRS + write window) lives in xbank_timers_fub today; the scheduler consumes the gate but does not pre-plan around it. A smarter scheduler could **batch by rank** (issue all ready reads on rank 0, then all on rank 1) to amortize tRTRS. This is a v2 feature; v1 takes the simpler per-cycle gating.
- Whether OBS_INORDER_FALLBACKS is useful for characterization is not yet validated; the counter is cheap, but if it’s never read by the bring-up team it can be dropped in v2.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

13 HAPPY Page Predictor (page_predictor_fub)

Module: page_predictor_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Synthesized only when PAGE_POLICY == HAPPY_HYBRID. See HAS §3.2 page_predictor for algorithm details (Ghasempour et al. 2015).

13.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

14 Bank Machine (bank_machine_fub)

Module: bank_machine_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Instantiated $\text{NUM_RANKS} \times \text{NUM_BANKS}$ times. See HAS §3.3 for per-bank FSM and refresh handshake.

14.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

15 Cross-Bank Timers (xbank_timers_fub)

Module: xbank_timers_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Per-rank tRRD/tFAW, global tCCD/tWTR/tRTW, cross-rank tRTRS/tCS. See HAS §3.3 xbank_timers and the Per-Rank vs. Cross-Rank Scope table.

15.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

16 Refresh Manager (refresh_mgr_fub)

Module: refresh_mgr_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Round-robin rank dispatch, DARP bank selection, postponer, periodic ZQCS, per-rank PASR mask. See HAS §3.4.

16.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)

- Verification notes (cocotb test plan)

17 Init Engine (`init_engine_fub`)

Module: `init_engine_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

Microprogram-driven cold-boot. Step-table ROM in `rtl/macro/` per memtype. See HAS §3.5.

17.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

18 Power-State FSM (`power_state_fub`)

Module: `power_state_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

Active / Active-PD / Self-Refresh / Deep-PD; per-rank CKE; SR coordination; quiet-point detector. See HAS §3.5.

18.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

19 Command Encoder (cmd_encoder_fub)

Module: cmd_encoder_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

DDR2 ras/cas/we vs LPDDR2 20-bit CA bus, selected at elaboration by MEMTYPE. See HAS §3.6.

19.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

20 DFI Gear (gear_dfi_fub)

Module: gear_dfi_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Per-phase packing per N_PHASES / WRPULSE / RDPULSE. See HAS §3.6 gear_dfi.

20.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

21 ODT Control (odt_ctrl_fub)

Module: odt_ctrl_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Per-rank dfi_odt[r] per ODT_RULE_MULTIRANK. The home of the cross-rank read/write ODT rules. See HAS §3.6.

21.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)

- Verification notes (cocotb test plan)

22 Write Data Path (wr_data_path_fub)

Module: wr_data_path_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

AXI W → DFI wrdata. Width conversion AXI DW → NP × DFI_DW. Strobe propagation. See HAS §3.7.

22.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

23 Read Data Path (rd_data_path_fub)

Module: rd_data_path_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

DFI rddata → AXI R. CL/CWL-aware capture, ID-tagged OoO completion. See HAS §3.7.

23.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

24 AXI4 Slave Protocol

Status: Skeleton — to be authored during MAS week

The AXI4 slave port carries host requests into the controller.

24.1 Scope of This Section

This section is the **wire-level** AXI4 contract specific to this controller — what's supported, what's relaxed, what's omitted. It is not an AXI4 tutorial; refer to ARM IHI 0022 for the full protocol.

24.2 Supported Features (v1)

- AW/W/B/AR/R 5-channel handshake
- INCR burst type (mandatory)
- ID-aware OoO completion (when `AXI_000_ACROSS_IDS = true`)
- 64 / 128 / 256-bit data widths
- 1–16-bit ID widths
- Per-ID in-order completion

24.3 Optional Features (build-time)

- FIXED burst (`SUPPORT_FIXED_BURST`)
- WRAP burst (`SUPPORT_WRAP_BURST`)

24.4 Unsupported Features

- AXI exclusive accesses
- AXI4 user signals (no USER_* widths)
- AXI4 cache-coherent extensions

24.5 TODO

- Per-channel timing diagrams (wavedrom)
- Backpressure semantics in detail
- Response-ordering examples for OoO-across-IDs
- Burst-boundary corner cases (4 KB boundary, address-alignment)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

25 DFI v2.1 Master Protocol

Status: Skeleton — to be authored during MAS week

The DFI v2.1 master port drives the DRAM PHY.

25.1 Scope

This section is the wire-level DFI v2.1 contract as implemented by this controller. The full DFI v2.1 spec covers more — training, frequency change, low-power negotiation — see HAS §2.1 for what is and isn't in scope here.

25.2 Sub-Interfaces Supported

- Command sub-interface
- Write-data sub-interface
- Read-data sub-interface
- Status sub-interface
- Update sub-interface (controller-initiated and PHY-initiated)

25.3 Sub-Interfaces Not Supported (v1)

- Training (not required at v2.1 for DDR2/LPDDR2)

- Frequency change
- Low-power negotiation (CKE used directly)
- CRC (not in DDR2/LPDDR2)

25.4 Phase / Gear Topology

N_PHASES per MC clock; gear FUB (see §2.15) handles packing.

25.5 TODO

- Per-signal timing diagrams (rise-to-rise)
- WRPHASE / RDPHASE selection guidelines
- ctrlupd / phyupd protocol detail (rare in v2.1 but spec-mandated)
- DDR2 ras/cas/we vs LPDDR2 CA encoding (cross-ref to §2.14)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

26 APB CSR Slave Protocol

Status: Skeleton — to be authored during MAS week

The APB CSR slave is the configuration and observation interface.

26.1 Protocol Compliance

APB3 (pslverr supported); APB4 (pstrb) optional.

26.2 Behavior

- Two-cycle access (SETUP + ENABLE) per the APB spec
- pslverr returns 1 for:
 - Writes to unimplemented schemes in ADDR_MAP_TUNING.scheme_or
 - Writes to ranks beyond NUM_RANKS (PASR registers, observation registers)
 - Writes to read-only registers
 - Reads from reserved addresses

- All registers are 32-bit; sub-32-bit accesses are aligned to 32-bit boundaries

26.3 CDC

The APB  MC CDC lives in this FUB. See §1.3 and §2.18 `csr_apb_fub` for the CDC topology.

26.4 TODO

- pslverr decision tree
- APB-side timing diagram
- Sub-32-bit access policy
- CDC implementation detail

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

27 Register Map

Status: Skeleton — to be authored during MAS week

The full CSR map is in HAS §6.3. This MAS section is the **implementation-level** view: how each register is generated, what the reset value generation rule is, how runtime overrides are staged, and the canonical CSR YAML source.

27.1 Source of Truth

The CSR map is generated from `regs/ddr2_lpddr2_csrs.yaml` (PeakRDL- or RDL-style YAML). The build runs the generator to produce:

- `regs/ddr2_lpddr2_csr_pkg.sv` — SV package with offsets, masks, reset values
- `regs/ddr2_lpddr2_csr_block.sv` — the wrapped register file
- `regs/ddr2_lpddr2_csrs.h` — C header for software
- `regs/ddr2_lpddr2_csr_block_uvm.sv` — UVM register model

The MAS does not re-publish the bit-level register table; it documents the **generation pipeline** and the **runtime semantics** (quiet-point staging, etc.).

27.2 Staging vs. Live State

Every R/W field has two storage cells:

1. **APB-side staging register** — written immediately on the APB write
2. **MC-side live register** — copied from staging on the next quiet point

The staging→live commit is one MC cycle. Reads can return either side depending on the field — observation fields (OBS_*) read live state; configuration fields (*_TUNING) read staging by default but switch to live after CTRL.config_apply is acknowledged.

27.3 TODO

- CSR YAML schema reference
- Generation script invocation
- Quiet-point handshake protocol detail
- Software access pattern (config-apply sequence)
- Generated UVM register model overview

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

28 Runtime Overrides and Quiet Points

Status: Skeleton — to be authored during MAS week

This section documents the runtime override mechanism — how a CSR write becomes a live datapath change without rebuilding or resetting.

28.1 Override Categories

Three flavors of override exist:

1. **Synthesized-MAX, runtime-active** — the silicon includes the maximum-capability logic; runtime CSR selects how much of it is active. Examples: SCHED_TUNING.lookahead_active, REFRESH_TUNING.refresh_defer_active.

2. **Synthesized-set, runtime-mux** — the silicon includes a discrete set of variants; runtime CSR picks one. Examples: `ADDR_MAP_TUNING.scheme_or`, `REFRESH_TUNING.refpb_policy_or`.
3. **Pure runtime** — no silicon-area cost beyond the register itself. Examples: `REFRESH_TUNING.zqcs_freq_hz`, `INIT_TUNING.zq_retries`.

28.2 Quiet-Point Sequence

1. Software writes the staging field via APB.
2. Software writes `CTRL.config_apply = 1`.
3. Controller drives `txn_queue` to drain, blocks new AXI accepts, completes any in-flight refresh.
4. `power_state_fub` asserts `quiet_point` for one cycle.
5. `csr_apb_fub` copies staging → live for every R/W field.
6. `STATUS.config_settled` rises.
7. Software reads `STATUS.config_settled`, clears `CTRL.config_apply`, and resumes traffic.

The drain phase is bounded — typically < 256 MC cycles for a healthy queue — and is the only operational disruption from a runtime override.

28.3 TODO

- Worst-case drain latency analysis
- Override propagation timing for each category
- Multi-write coalescing (if multiple staging writes are queued, commit them all at one quiet point)
- Software examples (init-time vs. runtime override patterns)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

29 Family-Wide Config-Bit Applicability

Status: Skeleton — to be authored during MAS week

The full family-wide config-bit registry is in HAS §5.4. This MAS section documents the implementation-level shape of those bits in **this controller specifically**.

29.1 Per-Bit Implementation Notes for DDR2 / LPDDR2

Bits flagged (reserved) in HAS §5.4 ECC_TUNING are present in the CSR map at the documented offsets but read as zero and writes are ignored. The CSR slave does not raise `pslverr` on these — software portability requires writing the family-wide config word to succeed on flavors that haven't implemented every bit.

Bits flagged DDR3+, DDR4+, LPDDR3+, LPDDR4-only likewise read as zero and ignore writes in this DDR2/LPDDR2 controller.

The `0xFF8` capability vector reports the build's actual capabilities; software queries this once at boot to decide which family-wide writes are meaningful.

29.2 Soft-N/A Behavior Examples

- Writing `SCHED_TUNING.bank_group_balance = 1` in DDR2 build: silently ignored, no `pslverr`. Read-back returns 0.
- Writing `POWER_TUNING.dpd_enable = 1` in DDR2 build: silently ignored. (DPD is LPDDR-only.)
- Reading `INIT_TUNING.cbt_enable` in any v1 controller: returns 0. (LPDDR4 only.)

29.3 TODO

- Per-bit implementation note for every family-wide bit
- Test plan for each soft-N/A bit
- Software bring-up sequence demonstrating capability-vector-gated writes

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

30 Initialization Sequence

Status: Skeleton — to be authored during MAS week

The init sequence from power-on through DRAM ready, as seen by software.

30.1 TODO

- Step-by-step software pseudocode
- Timing of each step (CSR write → controller state advance)
- Error-detection at each step
- Multi-rank init differences (per-rank ZQ, per-rank MR loads)
- Sim shortcut path (SIM_INIT_SCALE > 1)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

31 Refresh and Power-State Programming

Status: Skeleton — to be authored during MAS week

Programming the refresh subsystem and the power-state FSM from software.

31.1 TODO

- Setting tREFI from CSR
- Selecting REFab vs REFpb per rank
- LPDDR2 PASR mask programming per rank
- Temperature-derate read-back (MR4 / TEMP_DERATE_RANK)
- ZQCS frequency tuning
- Entering / exiting self-refresh
- Entering / exiting Deep Power Down (LPDDR2)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

32 Multi-Rank Programming

Status: Skeleton — to be authored during MAS week

The multi-rank-specific programming concerns: per-rank MR loads, per-rank PASR, cross-rank ODT, refresh fairness.

32.1 TODO

- Per-rank MR0/1/2/3 programming sequence
- Per-rank PASR programming
- ODT rule selection (ODT_RULE_MULTIRANK)
- Cross-rank ODT verification with the BFM
- Rank-disable semantics (RANK_TUNING.rank_enable_mask)
- Multi-rank refresh schedule observation

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

33 Error Handling

Status: Skeleton — to be authored during MAS week

Error reporting paths and the software response model.

33.1 TODO

- Error categories: init failure, refresh-deadline miss, AXI queue overflow, DFI rddata-valid timeout
- IRQ topology (irq_init_done, irq_init_error, irq_overflow)
- Clearing latched errors via CSR
- Diagnosis sequence: history register (STATUS_HISTORY), debug observation FUBs
- Software recovery patterns (re-init, soft-reset, full reset)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

34 Build-Time Configuration Reference

Status: Skeleton — to be authored during MAS week

The full build-time parameter table is in HAS §5.2. This MAS section is the **build-script** view: how each parameter is wired through filelists, includes, and synthesis scripts.

34.1 TODO

- Parameter override on synthesis command line
- Filelist composition per parameter combination
- `rtl/includes/ddr2_lpddr2_config.svh` macro definitions
- Conditional FUB inclusion (e.g., `page_predictor` only when `PAGE_POLICY == HAPPY_HYBRID`)
- Per-memtype filelists

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

35 Runtime Configuration Reference

Status: Skeleton — to be authored during MAS week

The full runtime CSR is in HAS §6.3. This MAS section is the **driver-author** view: which fields can be tuned at runtime, in what order, and with what side-effects.

35.1 TODO

- Field-by-field runtime tuning cookbook
- Recommended bring-up sequence
- Recommended characterization sweep sequence
- Telemetry registers driving each tuning decision